



PASO AULA

Documentación del programador

Manuel Duque Núñez
mduquenu@alumnos.unex.es

Índice.

Índice.	0
Clase Window.	2
Métodos.-	2
Atributos.-	3
Archivo de configuración “.json” por defecto.	4
Clase State.	4
Clase Processor.	5
Métodos.-	5
Atributos.-	6

Clase Window.

La clase **Window** utiliza una función decorador para permitir una única instancia de la clase, y contiene lo necesario para ejecutar la interfaz de usuario del programa y todos los eventos relacionados. Para ello, se compone de:

Métodos.-

- **__init__()**: Constructor de la clase Window.
- **__speedButton__()**: Es un **método privado** que permite **cambiar la velocidad** del vídeo. Comprueba que la velocidad esté en el rango correcto mediante el archivo de configuración "config.json".
- **__startDebug__()**: Es un **método privado** que permite **mostrar** todas las ventanas necesarias para **la depuración**. Hace uso de la clase Processor.
- **__stopDebug__()**: **Método privado** que permite **detener la depuración** y cerrar todas las ventanas auxiliares de la depuración.
- **__pauseButton__()**: **Método privado** que permite **pausar** el vídeo o **continuar reproduciendo**. Comprueba si se estaba reproduciendo para pararlo, y en caso de que estuviera parado lo continúa reproduciendo con la velocidad anterior.
- **__restartButton__()**: Es un **método privado** que gestiona el reinicio del vídeo. **Restablece todos los valores por defecto**, incluyendo los niveles de las barreras y contadores.
- **__set_default_barriers__(lower, upper)**: **Método privado** que permite **establecer las barreras por defecto**. La barrera inferior nunca puede superar a la barrera superior y viceversa.
- **__change_barrier_lower_value__(lower, color, thickness)**: Es un **método privado** que gestiona el evento lanzado **cuando** el usuario **cambia el valor de la barrera inferior**. Valida que su altura, desde el punto de vista de la imagen, está por debajo o es igual al de la barrera superior, y lo actualiza y dibuja sobre la imagen. **Permite cambiar** la barrera con el vídeo **en estado de pausa** recuperando el último frame.
- **__change_barrier_upper_value__(upper, color, thickness)**: Es un **método privado** que gestiona el evento lanzado **cuando** el usuario **cambia el valor de la barrera superior**. Valida que su altura, desde el punto de vista de la imagen, está por encima o es igual al de la barrera inferior, y lo actualiza y dibuja sobre la imagen. **Permite cambiar** la barrera con el vídeo **en estado de pausa** recuperando el último frame.
- **__update__()**: Es un **método privado** encargado de gestionar la actualización de la UI por cada intervalo de tiempo definido sobre el reloj de la aplicación. Obtiene un frame, lo procesa con ayuda de la instancia de Processor, y lo adapta y muestra en el viewer.
- **__show_debug__(title, image, adapt)**: Es un **método privado** que funciona como **callback para que el "processor" haga el debug** correctamente. **Permite mostrar una ventana** con "cv2.imshow" (OpenCV) **con el título y la imagen** dada. Además, permite definir si se quiere adaptar la imagen al tamaño del viewer, sabiendo que se mostrarán también sobre dicha imagen las referencias de las barreras.

- **__adapter_to_video_source__(image, centroid): Método privado** que **permite adaptar una imagen** de tipo "numpy.ndarray" o un **centroide** de tipo "tuple" **al viewer**. Devuelve la conversión de tipo Any. Si se trata de una imagen, se adaptarán y mostrarán sobre la imagen las referencias de las barreras.
- **__set_image_to_view_source__(image):** Es un **método privado** que **establece en el "video_source"** de la UI **la imagen** dada de tipo "numpy.ndarray".
- **__calculate_position_barriers__(image):** Es un **método privado** que **calcula la posición de las barreras en la imagen** dada. Hace la operación "100 - {valor barrera}" para invertir el valor (el rango de valores de la configuración debería de ir de 0 a 99 de abajo hacia arriba). Una vez invertida la posición de la barrera para poder tomar la referencia de arriba hacia abajo, se **calcula la posición de cada** una según la imagen, **haciendo una regla de tres respecto a la altura** de la imagen. Así, las barreras estarán escaladas a la altura de la imagen. Devuelve las posiciones en el eje "y" de cada barrera.
- **__show_lines__(image):** Es un **método privado** que **dibuja las barreras** en la imagen dada. Para ello calcula la posición de las barreras y las dibuja en la la imagen.

Atributos.-

- **utils:** Objeto de la clase Utils. Permite cargar archivos de configuración "json" y gestionar la lectura de los diccionarios.
- **video:** Vídeo cargado mediante OpenCV "cv2.VideoCapture". Carga el archivo configurado en el archivo de configuración "config.json".
- **processor:** Objeto de la clase Processor. Permite gestionar el funcionamiento de la lógica de la aplicación, añadiendo una capa de abstracción y separando la UI de la lógica.
- **ui:** Diseño de Qt cargado. Es la instancia de la interfaz del usuario.
- **timer:** Timer de Qt para actualizar la interfaz de usuario.
- **debug:** Bandera booleana para saber si está activado el modo depuración o no.
- **firstFrame:** Frame o imagen de tipo "numpy.ndarray" en gris sobre la que se comparan el resto de frames para detectar el movimiento.
- **__lower_barrier__:** Diccionario con la información de la barrera inferior. Almacena la posición en el eje Y como "y", el color como "color" y el thickness como "thickness". La posición del eje Y no está adaptada a la imagen.
- **__upper_barrier__:** Diccionario con la información de la barrera superior. Almacena la posición en el eje Y como "y", el color como "color" y el thickness como "thickness". La posición del eje Y no está adaptada a la imagen.
- **__barriers__:** Tupla de dos valores que contiene la posición real en la imagen de la barrera inferior y superior respectivamente. Sólo almacena el valor en el eje Y de cada una de ellas.

Archivo de configuración “.json” por defecto.

NOTA: La aplicación debería de ser llamada desde la raíz del proyecto, al mismo nivel que el archivo “mainwindow.ui” o “main.py”.

```
1  {
2      "video_relative_path": "video.wmv",
3      "FPS": 60,
4      "MAX_SPEED": 100,
5      "ui": {
6          "ui_relative_path": "mainwindow.ui",
7          "window_title": "Paso aula",
8          "counter_text": "People: {0}"
9      },
10     "barriers": {
11         "upper": {
12             "y": 35,
13             "color": [0, 255, 0],
14             "thickness": 5
15         },
16         "lower": {
17             "y": 22,
18             "color": [255, 0, 0],
19             "thickness": 5
20         }
21     }
22 }
```

Clase State.

Es una clase que se hereda de la clase Enum, y se usa para disponer de los diferentes estados por los que puede pasar el programa. Hace más legible y fácil de entender el código. Contiene los valores:

```
1  class State(enum.Enum):
2      '''
3      Define the states of the centroid position in the image. The states are:
4      - INSIDE: The centroid is above the upper barrier.
5      - OUTSIDE: The centroid is below the lower barrier.
6      - ENTERING: The centroid is between the barriers and is going to the inside.
7      - EXITING: The centroid is between the barriers and is going to the outside.
8      '''
9      INSIDE = 1
10     OUTSIDE = 2
11     ENTERING = 3
12     EXITING = 4
```

Clase Processor.

La clase **Processor** utiliza una función decorador para permitir una única instancia de la clase, y contiene lo necesario para ejecutar la lógica del programa. Para ello, se compone de:

Métodos.-

- **`__init__()`**: Es el constructor de la clase Processor. Toma la instancia de la clase Utils.
- **`debug(callback)`**: Utiliza la función de **callback** para mostrar el **contenido de las imágenes generadas en el procesamiento de la imagen**. Hace la llamada de callback para mostrar las ventanas de: "gray", "frameDelta", "blurred", "thresh" y "frame".
- **`stop_debug()`**: Utilizado cuando se desea detener la ejecución del método de debug.
- **`restart()`**: Permite restablecer el estado del procesador ("processor" -o "self"-). Restablece el contador, el estado inicial y el estado del frame procesado.
- **`get_last_frame()`**: Permite recuperar el último frame procesado.
- **`__from_frame_to_contours__(image, comparative_image)`**: **Método privado** que permite obtener los contornos del objeto en movimiento. La "comparative_image" debe de estar en gris. Aplica los filtros necesarios a la imagen "image" y las compara. Aplica: "cv2.cvtColor", "cv2.absdiff", "cv2.GaussianBlur", "cv2.threshold", "cv2.findContours" y "imutils.grab_contours". Almacena el resultado de todo este proceso y los diferentes filtros en la variable "__frame_state__", junto al "frame" original ("image").
- **`__from_contours_to_centroid__(contours)`**: Obtiene el centroide del contorno dado.
- **`update(frame, comparative_reference)`**: Es llamado cuando un nuevo frame se ha leído de la cámara o vídeo. Recibe el frame leído y la referencia para compararlos. Devuelve un diccionario con la clave "frame" y de valor el frame, y la clave "centroid" y el valor del centroide.
- **`process_centroid(centroid, barriers)`**: Procesa la ubicación del centroide en comparación con las barreras. Devuelve el contador de personas que hay en el interior.
- **`__update_counter__(increment)`**: Es un **método privado** que permite **actualizar el contador de forma positiva o negativa**. Aplica un **mínimo de 0**, tras lo cual no puede decrementarse más. Devuelve el nuevo valor del contador.
- **`__centroid_between_barriers__(centroidY, upper_barrier, lower_barrier)`**: Es un **método privado** que comprueba y gestiona el estado del programa si un centroide está **entre ambas barreras**.

- **__centroid_below_lower_barrier__()**: Es un **método privado** que comprueba y gestiona el estado del programa si el centroide está por **debajo de la barrera inferior**.
- **__centroid_above_upper_barrier__()**: Es un **método privado** que comprueba y gestiona el estado del programa si el centroide está **por encima de la barrera superior**.

Atributos.-

- **utils**: Objeto de la clase Utils. Permite cargar archivos de configuración "json" y gestionar la lectura de los diccionarios.
- **__frame_state__**: Diccionario que almacena el procesamiento de la imagen dada. Contiene la clave "frame" para la imagen procesada y la clave "state" para el estado. El estado es un diccionario que contiene las claves: "gray", "frameDelta", "blurred", "thresh", "cnts" y "contours".
- **__counter__**: Contador de tipo entero de las personas que hay en el interior del aula.
- **__state__**: Estado del último movimiento de la persona. Es del tipo enumerado "State", e inicialmente vale State.INSIDE.